

OCL Constraints

Assets Tracking System — SE2

context User

INVARIANTS

```
inv UsernameNotBlank: self.username <> null and self.username.size() >= 3
inv ValidEmail: self.email <> null and self.email.indexOf('@') > 0
inv PasswordNotEmpty: self.password <> null and self.password.size() > 0
inv AtLeastOneRole: self.roles->notEmpty()
inv CreatedBeforeUpdated: self.updatedAt >= self.createdAt
inv UniqueEmail: User.allInstances()->isUnique(u | u.email)
inv UniqueUsername: User.allInstances()->isUnique(u | u.username)
inv InactiveNoActiveAssignments:
not self.active implies
self.assignments->forAll(a | not a.active)
```

OPERATIONS

register(username,email,password): User

```
pre username <> null and username.size() >= 3
pre email <> null and email.indexOf('@') > 0
pre password <> null and password.size() >= 6
pre User.allInstances()->forAll(u | u.email <> email)
pre User.allInstances()->forAll(u | u.username <> username)
post result <> null and result.active = true
post result.email = email and result.username = username
```

changePassword(oldPassword, newPassword): void

```
pre oldPassword <> null and newPassword.size() >= 6
pre oldPassword <> newPassword
post self.password <> oldPassword@pre
```

deactivate(): void

```
pre self.active = true
post self.active = false
post self.assignments->forAll(a | not a.active)
```

updateProfile(username, email): User

```
pre username <> null and username.size() >= 3
pre email <> null and email.indexOf('@') > 0 and self.active = true
post self.username = username and self.email = email
post self.updatedAt > self.updatedAt@pre
```

hasRole(roleName): boolean

```
pre roleName <> null and roleName.size() > 0
post result = self.roles->exists(r | r.name.toString() = roleName)
```

context Role

INVARIANTS

```
inv ValidRoleName: self.name in {ADMIN, ASSET_MANAGER, EMPLOYEE}
inv UniqueRoleName: Role.allInstances()->isUnique(r | r.name)
```

OPERATIONS

assignToUser(user): void

```
pre user <> null and user.active = true
pre not user.roles->includes(self)
post user.roles->includes(self)
```

revokeFromUser(user): void

```
pre user.roles->includes(self)
pre user.roles->size() > 1
post not user.roles->includes(self)
```

context AssetCategory

INVARIANTS

```
inv CategoryNameNotBlank:
self.name <> null and self.name.size() > 0
inv UniqueCategoryName:
AssetCategory.allInstances()->isUnique(c | c.name)
```

OPERATIONS

create(name, description): AssetCategory

```
pre name <> null and name.size() > 0
pre AssetCategory.allInstances()->forAll(c | c.name <> name)
post result.name = name and result.description = description
```

update(name, description): AssetCategory

```
pre name <> null and name.size() > 0
post self.name = name and self.description = description
```

delete(): void

```
pre self.assets->isEmpty()
```

context Asset

INVARIANTS

```
inv AssetNameNotBlank: self.name <> null and self.name.size() > 0
inv UniqueSerialNumber: Asset.allInstances()->isUnique(a | a.serialNumber)
inv ValidAssetStatus:
  self.status in {AVAILABLE, ASSIGNED, UNDER_MAINTENANCE, RETIRED}
inv AssignedHasActiveAssignment:
  self.status = ASSIGNED implies
  self.assignments->select(a | a.active)->size() = 1
inv AvailableHasNoActiveAssignment:
  self.status = AVAILABLE implies
  self.assignments->forAll(a | not a.active)
inv RetiredNotAssigned:
  self.status = RETIRED implies
  self.assignments->forAll(a | not a.active)
inv WarrantyAfterPurchase:
  self.warrantyExpiry <> null implies
  self.warrantyExpiry > self.purchaseDate
inv MaxOneActiveAssignment:
  self.assignments->select(a | a.active)->size() <= 1
```

OPERATIONS

```
create(name, serialNumber, category, location): Asset
  pre name <> null and name.size() > 0
  pre serialNumber <> null and Asset.allInstances()->forAll(a | a.serialNumber <>
    serialNumber)
  pre category <> null and location <> null
  post result.status = AssetStatus::AVAILABLE
  post result.name = name and result.serialNumber = serialNumber
changeStatus(newStatus): void
  pre newStatus <> self.status
  pre self.status <> AssetStatus::RETIRED -- terminal state
  post self.status = newStatus
assign(user): AssetAssignment
  pre self.status = AssetStatus::AVAILABLE
  pre user <> null and user.active = true
  post self.status = AssetStatus::ASSIGNED
  post self.assignments->exists(a | a.active = true and a.returnedAt = null)
returnAsset(): void
  pre self.status = AssetStatus::ASSIGNED
  post self.status = AssetStatus::AVAILABLE
  post self.assignments->select(a | a.active)->isEmpty()
delete(): void
  pre self.status <> AssetStatus::ASSIGNED
```

```
pre self.maintenanceTickets->forall(t |  
  t.status = TicketStatus::CLOSED or t.status = TicketStatus::RESOLVED)  
isAvailable(): boolean  
post result = (self.status = AssetStatus::AVAILABLE)
```

context AssetAssignment

INVARIANTS

```
inv AssignedAtNotNull: self.assignedAt <> null  
inv ReturnedAfterAssigned:  
  self.returnedAt <> null implies self.returnedAt > self.assignedAt  
inv ActiveNoReturn: self.active implies self.returnedAt = null  
inv InactiveHasReturn: not self.active implies self.returnedAt <> null  
inv SingleActivePerAsset:  
  AssetAssignment.allInstances()  
  ->select(a | a.asset = self.asset and a.active)->size() <= 1
```

OPERATIONS

assign(asset, user): AssetAssignment

```
pre asset.status = AssetStatus::AVAILABLE  
pre user <> null and user.active = true  
pre AssetAssignment.allInstances()->select(a | a.asset = asset and a.active)->isEmpty()  
post result.active = true and result.returnedAt = null  
post asset.status = AssetStatus::ASSIGNED
```

returnAsset(): void

```
pre self.active = true  
post self.active = false  
post self.returnedAt <> null  
post self.asset.status = AssetStatus::AVAILABLE
```

isActive(): boolean

```
post result = self.active
```

context MaintenanceTicket

INVARIANTS

```
inv TicketTitleNotBlank: self.title <> null and self.title.size() > 0
inv ValidTicketStatus:
  self.status in {OPEN, IN_PROGRESS, RESOLVED, CLOSED}
inv ValidPriority: self.priority in {LOW, MEDIUM, HIGH}
inv ResolvedAtOnlyWhenResolved:
  self.resolvedAt <> null implies
  self.status in {RESOLVED, CLOSED}
inv ClosedHasResolvedAt:
  self.status = TicketStatus::CLOSED implies self.resolvedAt <> null
inv AssetUnderMaintenanceWhenOpen:
  self.status in {OPEN, IN_PROGRESS} implies
  self.asset.status = AssetStatus::UNDER_MAINTENANCE
```

OPERATIONS

create(asset,user,title,description,priority): MaintenanceTicket

```
pre asset <> null and user <> null and user.active = true
pre title <> null and title.size() > 0
pre asset.status <> AssetStatus::RETIRED
post result.status = TicketStatus::OPEN
post asset.status = AssetStatus::UNDER_MAINTENANCE
```

isValidTransition(currentStatus, newStatus): boolean

```
post result = (
  (currentStatus = OPEN and newStatus = IN_PROGRESS) or
  (currentStatus = IN_PROGRESS and newStatus = RESOLVED) or
  (currentStatus = RESOLVED and newStatus = CLOSED) )
```

updateStatus(newStatus): void

```
pre self.status <> newStatus
pre self.isValidTransition(self.status, newStatus)
post self.status = newStatus
post self.updatedAt > self.updatedAt@pre
```

resolve(resolutionNote): void

```
pre self.status <> TicketStatus::RESOLVED and self.status <> TicketStatus::CLOSED
pre resolutionNote <> null and resolutionNote.size() > 0
post self.status = TicketStatus::RESOLVED
post self.resolvedAt <> null
```

close(): void

```
pre self.status = TicketStatus::RESOLVED
post self.status = TicketStatus::CLOSED
```

addNote(content, author): TicketNote

```
pre content <> null and content.size() > 0
pre author <> null and author.active = true
pre self.status <> TicketStatus::CLOSED
```

```
post self.notes->size() = self.notes@pre->size() + 1
post self.notes->exists(n | n.content = content)
```

context TicketNote

INVARIANTS

```
inv NoteContentNotBlank: self.content <> null and self.content.size() > 0
inv NoteCreatedAtNotNull: self.createdAt <> null
inv NoteAfterTicketCreation:
  self.createdAt >= self.ticket.createdAt
```

OPERATIONS

create(content, ticket, author): TicketNote

```
pre content <> null and content.size() > 0
pre ticket <> null and ticket.status <> TicketStatus::CLOSED
pre author <> null and author.active = true
post result.content = content and result.ticket = ticket
```

context AuditLog

INVARIANTS

```
inv AuditActionNotBlank: self.action <> null and self.action.size() > 0
inv AuditEntityTypeNotBlank: self.entityType <> null and self.entityType.size() > 0
inv AuditEntityIdPositive: self.entityId > 0
inv AuditPerformedAtNotNull: self.performedAt <> null
inv AuditActorNotNull: self.actor <> null
```

OPERATIONS

log(action, entityType, entityId, oldValue, newValue, actor): AuditLog

```
pre action <> null and entityType <> null
pre entityId > 0 and actor <> null
post result.action = action and result.performedAt <> null
post result.entityId = entityId and result.actor = actor
```

getByDateRange(from, to): List

```
pre from <> null and to <> null and from <= to
post result->forall(l | l.performedAt >= from and l.performedAt <= to)
```

context Notification

INVARIANTS

```
inv NotificationMessageNotBlank:
  self.message <> null and self.message.size() > 0
inv ValidNotificationType:
  self.type in {ASSIGNMENT, MAINTENANCE, SYSTEM}
inv NotificationCreatedAtNotNull: self.createdAt <> null
```

OPERATIONS

send(recipient, message, type): Notification

```
pre recipient <> null and recipient.active = true
```

```
pre message <> null and message.size() > 0
pre type in {ASSIGNMENT, MAINTENANCE, SYSTEM}
post result.readStatus = false
post result.message = message and result.recipient = recipient
markAsRead(): void
pre self.readStatus = false
post self.readStatus = true
getUnreadByUser(user): List
pre user <> null
post result->forAll(n | n.readStatus = false and n.recipient = user)
```